API Gateway Introduction and Types

Explore Amazon API Gateway to understand its different API types—REST, HTTP, and WebSocket—and how to configure them for secure, scalable serverless applications. Discover practical examples and use cases to help you design and deploy APIs that connect AWS services effectively.

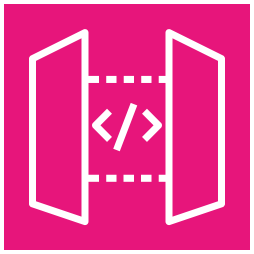
Amazon API Gateway is the developer's entry point into AWS-powered APIs. Whether we're building RESTful microservices, low-latency backends, or real-time chat apps, API Gateway simplifies connectivity, security, and traffic management, all without managing infrastructure.

In the AWS Certified Developer – Associate (DVA-C02) exam, we're expected to design and implement application workflows that use Amazon API Gateway to expose and manage backend functionality. We'll learn how to set up different types of APIs (REST, HTTP, or WebSocket) and how to decide which type best fulfills our needs. These are core developer skills tested on the exam and critical for building real-world serverless and event-driven applications on AWS.

What is Amazon API Gateway?

Amazon API Gateway allows us to expose REST, HTTP, or WebSocket endpoints and route them to AWS services like Lambda, DynamoDB, and more. It abstracts the complexity of infrastructure provisioning, authorization, transformation, throttling, caching, and monitoring.

Ask



Rather than managing API servers ourselves, we rely on API Gateway to handle request validation, traffic management, and endpoint scaling. It also integrates natively with IAM, Cognito, CloudWatch, and X-Ray, allowing us to build secure and observable APIs.

Why developers use API Gateway

API Gateway is central to building decoupled, event-driven, and serverless applications on AWS. We can easily connect frontend clients to Lambda functions, configure different behaviors per environment using stage variables, and enforce authorization rules with minimal custom code.

From a developer's standpoint, API Gateway helps us:

- Rapidly prototype and launch APIs.
- Apply throttling and usage controls.
- Define validation rules and transformations.
- Monitor performance and errors.

Because it integrates with other AWS services, API Gateway fits naturally into CI/CD workflows and infrastructure as code templates like AWS SAM or CloudFormation.

Types of APIs in API Gateway

API Gateway supports three API types:

Ask

1. REST APIs

These APIs are feature-rich, highly configurable, and ideal when we need fine-grained control, custom authorizers, caching, usage plans, and full request/response transformation. Here's how we can create a REST API using the AWS CLI:

main.sh Shell

```shell
aws apigateway create-rest-api \
  --name my-rest-api \
  --description "A REST API"
```

Create a REST API using the AWS CLI

This command creates a new REST API with default regional endpoint settings.

Request and response mapping templates

REST APIs support request and response mapping templates, which allow us to transform payloads at the API Gateway level using Velocity Template Language (VTL). This means we can adapt client requests to match backend expectations, or vice versa, without changing backend code.

For example, we can take an incoming JSON payload like this:

Ask

```
1 { "name": "Alice", "age": 30 }
```

Incoming payload

and transform it to match the backend's expected input:

```
{ "username": "Alice", "user_age": 30 }
```

Expected input

by applying this request mapping template:

```
{
  "username": "$input.path('$.name')",
  "user_age": $input.path('$.age')
}
```

Mapping template

Similarly, we can modify the backend's response before it reaches the client, such as wrapping it in a custom envelope or filtering fields. Mapping templates are essential when working with legacy systems, third-party APIs, or when strict separation between frontend and backend concerns is required.

2. HTTP APIs

HTTP APIs are simpler, more cost-efficient and best suited for modern serverless applications that use OAuth2/JWT authorization and simple routing. They offer low latency and fast deployment, but lack some advanced REST API features. Here's how we can create an HTTP API using the AWS CLI:

main.sh Shell

```
aws apigateway create-api \
  --name my-http-api \
  --protocol-type HTTP \
  --target arn:aws:lambda:<Region>:<Account-id>:function:function
```

Ask

Create a REST API using the AWS CLI

This command creates an HTTP API that directly integrates with a specified Lambda function.

3. WebSocket APIs

Support bidirectional, real-time communication. Used in apps like chat systems, games, or collaborative tools where persistent connections are essential. The CLI command to create a WebSocket API is given below:

main.sh Shell

```
aws apigateway create-api \
  --name my-websocket-api \
  --protocol-type WEBSOCKET \
  --route-selection-expression '$request.body.action'
```

Create a REST API using the AWS CLI

Ask

This command creates a WebSocket API that uses the `action` field in the request body to route events.

Each API type serves different needs. We must understand their capabilities, limitations, and pricing to make informed decisions.

Completing API setup

We saw how to create different types of APIs. After creating an API, the next step is to configure how it behaves. This includes defining which routes or resources it supports, how requests are handled, and which backend service is called in response. Let's look at how to completely setup each of those APIs.

HTTP APIs

HTTP APIs use a route-based model where each route corresponds to a specific method and path, such as `GET /users`.

To define a route, we first create an integration that tells API Gateway which backend to call. For example, to integrate with a Lambda function:

main.sh C++

```
aws apigateway create-integration \
  --api-id $API_ID \
  --integration-type AWS_PROXY \
  --integration-uri arn:aws:lambda:<Region>:<Account-ID>:function:<La
  --payload-format-version 2.0
```

Ask

Create a Lambda integration

This creates a Lambda proxy integration using the newer HTTP API payload format.

Next, we define a route that uses this integration:

main.sh C++

```
aws apigateway create-route \
  --api-id $API_ID \
  --route-key "GET /users" \
  --target integrations/$INTEGRATION_ID
```

Define a route

With the route and integration in place, we need to deploy the configuration to a stage:

main.sh C++

```
aws apigateway create-deployment \
  --api-id $API_ID
```

Ask

Deploy the API

Then we create or update a stage to associate it with the deployment:

main.sh C++

```
aws apigateway create-stage \
  --api-id $API_ID \
  --stage-name prod \
  --deployment-id $DEPLOYMENT_ID
```

Create stage for the API

> **Note:** We'll learn more about integrations and stages in the next lesson.

REST APIs

REST APIs follow a resource and method structure rather than route keys. To expose an endpoint like `GET /users`, we first create a resource under the root resource:

Ask

main.sh C++

```
aws apigateway create-resource \
  --rest-api-id $REST_API_ID \
```

```
  --parent-id $ROOT_ID \
  --path-part users
```

Create resources

Then we define the method on that resource:

main.sh C++

```
aws apigateway put-method \
  --rest-api-id $REST_API_ID \
  --resource-id $RESOURCE_ID \
  --http-method GET \
  --authorization-type NONE
```

Define the methods

Next, we bind that method to a backend resource:

Ask

main.sh C++

```
aws apigateway put-integration \
```

```
  --rest-api-id $REST_API_ID \
  --resource-id $RESOURCE_ID \
  --http-method GET \
  --type AWS_PROXY \
  --integration-http-method POST \
  --uri arn:aws:lambda:<Region>:<Account-ID>:function:<Lambda-functio
```

Bind the backend resource

We've assumed a Lambda function to be the backend resource in this case.

Finally, we deploy the changes to a stage:

main.sh C++

```
aws apigateway create-deployment \
  --rest-api-id $REST_API_ID \
  --stage-name prod
```

Deploy the changes

Ask

WebSocket APIs

WebSocket APIs use named routes for lifecycle events like `$connect`, `$disconnect`, and `$default`. As with HTTP APIs, we begin by creating a Lambda integration:

main.sh C++

```
aws apigateway create-integration \
  --api-id $WS_API_ID \
  --integration-type AWS_PROXY \
  --integration-uri arn:aws:lambda:<Region>:<Account-ID>:function:<La
```

Create a Lambda integration

Then we define a route for the `$connect` event:

main.sh C++

```
aws apigateway create-route \
  --api-id $WS_API_ID \
  --route-key "$connect" \
  --target integrations/$INTEGRATION_ID
```

Ask

Define a route for connect

And finally, deploy the API:

main.sh C++

```
aws apigateway create-deployment \
  --api-id $WS_API_ID
```

Deploy the API

We also need a stage:

```
aws apigateway create-stage \
  --api-id $WS_API_ID \
  --stage-name prod \
  --deployment-id $DEPLOYMENT_ID
```

Add stage

Choosing the right API type

Ask

If we need full lifecycle management, validation, usage plans, and caching, we choose REST APIs. If speed and simplicity matter most, especially for Lambda

triggers, HTTP APIs are better. For streaming data or real-time messaging, WebSocket APIs are ideal.

The exam may present a scenario like: “A developer needs to send low-latency requests to a Lambda function with minimal setup.” The correct choice would be an HTTP API. Similarly, for an app requiring live updates to connected users, we would choose a WebSocket API.

> **Note:** While both REST and HTTP APIs support access logging to CloudWatch, only REST APIs support execution logging, which includes detailed information about request processing stages, integration latency, and errors. This makes REST APIs more suitable when deeper observability and troubleshooting capabilities are required.

Use cases and architectural benefits

API Gateway enables developers to implement a variety of architectural patterns that enhance scalability, reduce operational complexity, and promote modular design.

Some common patterns include:

Ask

- CRUD APIs backed by Lambda and DynamoDB
- Microservice entry points in a distributed system

- Mobile and web backends usin Cognito authentication
- Real-time APIs with persistent WebSocket connections
- Fan-out event processing using Step Functions or SNS
- Edge-optimized or private APIs based on network requirements

These patterns empower developers to offload common infrastructure concerns such as rate limiting, input validation, retry logic, and authentication. To support these patterns, API Gateway integrates with several AWS services:

- Lambda (compute logic)
- Cognito (user auth)
- DynamoDB (storage)
- S3 (static files or binary content)
- Step Functions (workflow orchestration)
- EventBridge (event routing)
- SQS and SNS (messaging)
- CloudWatch and X-Ray (observability)

Knowing how these services interact helps us design APIs that are secure, efficient, and maintainable.

Common pitfalls

Here are some common mistakes related to using API Gateway:

- Using REST APIs where HTTP APIs would suffice increases cost unnecessarily
- Forgetting to apply request or response mapping templates, leading to backend errors
- Choosing WebSocket APIs for workloads that don't need real-time, bidirectional communication

Ask

We should validate API requirements early to avoid configuration sprawl and choose the simplest working solution.

Conclusion

Amazon API Gateway is a powerful, fully managed gateway service that supports multiple API styles. We use it to front microservices, manage client access, and build scalable applications without managing servers.

Ask